Which inverse, and which derivative?

A new-contract audit of Asymptotic

Exact counterexamples, bounded reference code, metadata repairs, and focused development proposals

Prepared for	Vladimir Reshetnikov
Repository	`VladimirReshetnikov/Asymptotic`
Reviewed revision	`8e859961d7d37f008b826f3a8cad406460271614`
Revision timestamp	10 September 2026, 04:00:14 UTC
Review date	10 September 2026
Exclusion baseline	Maintained records for 36 prior reviews across four waves; 231 attributed entries, not 231 distinct defects
Executed evidence	28 independent Python test methods; exact polynomial oracles and separately identified numerical samples
Native execution	No Wolfram or Mathics package execution in this review

Contribution. A numerical root can satisfy the equation, the selected source side, and even the expected derivative sign without belonging to the inverse germ being approximated. Separately, a special-inverse adapter exposes a derivative-bound expression whose natural original-equation interpretation fails under its admitted target scaling. This report develops those two issues without recycling the earlier review backlog.

Abstract

This review analyzes an immutable revision of the Asymptotic repository, using its extensive existing review register as an exclusion baseline. The new findings concern two distinct boundaries between symbolic mathematics and user-facing evidence. First, the general numerical inverse checker seeds an unbracketed root search from the approximation and accepts a real root on the selected source side, subject to the retained source predicate. That does not identify the endpoint-incident inverse branch. Explicit cubic and quartic examples make the approximation an exact root on a different component, allowing an equation-root discrepancy of zero while the selected inverse has a nonzero error. The quartic also defeats a proposed repair based only on the derivative's sign. An exact rational-polynomial reference implementation selects the first critical-point-free source component and supplies independent regression oracles.

Second, the Gamma/LogGamma special-inverse adapter stores a LogGamma derivative lower bound under a key named `OriginalDerivativeLowerBound`, while the stored original function may be an arbitrarily scaled Gamma or LogGamma expression. A half-scaled LogGamma example proves that the untransported expression is not a lower bound for either the signed derivative or its magnitude. The report gives the corrected chain-rule factors, separates transformed and original equations, and supplies an opt-in Wolfram metadata repair. These are a diagnostic branch-identification gap and a concrete metadata-scope defect, not demonstrations of incorrect formal inverse coefficients or broken rational interval arithmetic. Native characterization and desired-contract tests are supplied but unexecuted; 28 independent Python methods were executed successfully.

Contents

1 Executive assessment **3**

2 Review method, coverage, and exclusion discipline **3**

2.1 Immutable source and actual execution 3

2.2 How nonduplication was checked 4

2.3 Source coverage and negative results 5

3 The contracts that meet at these two boundaries **5**

3.1 A formal inverse, a numerical root, and an error estimate are different objects 5

3.2 An equation transformation also transforms its derivative 6

4 N1: equation-root agreement can select the wrong inverse **6**

4.1 Source trace 6

4.2 An exact cubic witness 6

4.3 Why checking the derivative sign is not enough 7

4.4 A scalable exact family 8

4.5 What the witness does and does not invalidate 8

4.6 A conservative repair with a precise scope 9

4.7 An immediate user-level control 9

5 N2: the derivative-bound field names the wrong scope **10**
5.1 The precise source mismatch . . . 10
5.2 A counterexample wholly inside the recorded source domain . . . 10
5.3 Why the transformed bound is valid . . . 11
5.4 Correct transport and explicit quantifiers . . . 11
5.5 Narrow metadata repair . . . 12
5.6 Consequences that are not claimed . . . 12

6 Implementation, cost, and UX tradeoffs **13**
6.1 What the branch reference implementation actually does . . . 13
6.2 Costs of exact branch identification . . . 13
6.3 Refusal is informative when its scope is precise . . . 13
6.4 The bound repair should not pretend to be an interval prover . . . 14

7 A focused development plan **14**

8 Artifacts and reproducibility **15**
8.1 Included files . . . 15
8.2 Independent execution . . . 16
8.3 Native characterization and desired contracts . . . 16
8.4 Build and visual checks . . . 17

9 Conclusion **17**

A Pinned source loci and coverage limits **17**

1 Executive assessment

The repository's strongest feature is the amount of mathematical context it retains: finite expressions, exact weights, local coordinates, remainder descriptions, assumptions, source equations, and reconstruction information. Inspection covered ordinary power–log algebra, inverse construction, extended scales, special-function adapters, exact interval evidence, numerical checks, and substantial parts of the Mathics compatibility layer. This is not a review of the README alone. Nevertheless, source inspection and independent mathematics are not substitutes for running the package, and the conclusions below preserve that distinction.[9, 13, 11, 10, 12, 22]

The present revision follows a large amount of review-driven hardening. Its four-wave catalog contains 36 supplied reports, and its maintained crosswalks attribute 231 entries before overlaps are grouped. Treating each historical entry as a current defect would be misleading; presenting the same resource, refinement, numerical-precision, or branch-normalization recommendations again would also violate this review's scope. The new report therefore has *two* findings, not an inflated list assembled from adjacent old concerns.[2, 3, 4, 5]

ID	Classification	New issue	Evidence status
N1	High priority for reference interpretation; not a formal-series defect	Same-side equation-root acceptance does not identify the endpoint-incident inverse. A self-confirming root can make the displayed discrepancy zero on the wrong branch.	Source deduction; exact counterexamples and Python oracle executed; public native call unrun.
N2	Medium priority; exposed bound metadata/API scope	The stored derivative lower bound is for the transformed LogGamma equation, not generally for the stored scaled original function.	Source fact and exact inequality; independent samples; Wolfram repair unrun.

Immediate recommendation. Separate the evidence "this point solves the original equation" from "this point belongs to the selected inverse branch." Then remove or explicitly scope the ambiguous derivative-bound key. Neither change requires replacing the power–log coefficient algorithms. The accompanying code provides a restricted executable branch oracle and a narrow, opt-in metadata repair rather than a speculative wholesale rewrite.

The prioritization is intentionally asymmetric. N1 can mislead a user who is using a numerical comparison to judge whether an approximation is accurate on its intended branch. N2 is already a mathematically invalid interpretation of an exposed field, but this review did not identify an internal consumer that uses that field to produce an incorrect certified interval. It would be unjustified to promote N2 into a demonstrated false-certificate vulnerability.

2 Review method, coverage, and exclusion discipline

2.1 Immutable source and actual execution

All repository statements refer to `8e859961d7d37f008b826f3a8cad406460271614`. The commit timestamp is 10 September 2026 at 04:00:14 UTC, or 9 September at 21:00:14 in America/Los_Angeles. The

article does not silently follow later changes to `main`. The source was obtained through the GitHub connector; the local build environment did not contain a complete repository checkout.[1]

An available Wolfram evaluator was attempted, including a primitive version/arithmetic query, but the service returned a connection error. Mathics was not installed in the local environment, and network installation was unavailable. Consequently, this review reports no fresh native package output, no upstream-suite pass rate, no fresh CI acceptance, and no package timing. The source-predicted outcomes in this article are not transcriptions of native observations.

The successfully executed work consists of the original Python code in the archive. It uses exact SymPy rational/algebraic operations for the polynomial witnesses and branch reference, and separately labeled mpmath samples for derivative comparisons. There are 28 `unittest` methods. Twelve scaled-polynomial family subcases and eighteen derivative-transport family/scale/point subcases occur *inside* those methods; they are not additional package tests. The generated evidence records actual interpreter/library versions and the final test transcript.

2.2 How nonduplication was checked

The maintained implementation register and the wave-3 and wave-4 intake records supplied the principal crosswalk. The review indexes and relevant original report openings/novelty discussions were also inspected, especially reports 13, 17, and 28, which treat numerical error resolution and runtime precision. This is a comparison against the maintained inventory and relevant original material, not a claim that every page of all 36 manuscripts was reread end to end.[3, 4, 5, 6, 7, 8]

The distinction between N1 and those numerical findings is substantive. The earlier numerical issues concern unresolved tiny discrepancies, lost source displacements, or unavailable requested precision. Here all crucial source points, targets, residuals, and errors are exact rational numbers with ordinary magnitudes. More digits cannot change which root belongs to the inverse branch. N2 concerns the identity of the equation differentiated, not the loss of a tail bound under truncation or a general request to add special-function certificates.

Item	Nearest existing register territory	Nonduplicated contribution
N1	C21; D05; wave-4 numerical precision	A reference root can be exact and well resolved yet belong to the wrong connected branch. The new predicate is endpoint-component membership, not additional precision or checked evaluation of `Normal`.
N1 controls	C05; C13	The witness uses clean assumptions and can satisfy its retained source predicate. It does not exploit ambient assumptions or an unproved composition domain.
N2	C22; X05; D02	The specific existing derivative-bound property is mis-scoped under target scaling. The repair transports an already available bound; it is not a general certificate-engine or API-registry proposal.
Excluded	Remaining maintained C/P/D/V/X and wave entries	Existing findings and roadmaps remain in their original reports. Their detailed recommendations are not reproduced here.

Absence from the inspected crosswalk is not a theorem about every sentence of historical prose. That

limitation is recorded in the novelty ledger. It does not justify silently reusing the old findings, and this report does not do so.

2.3 Source coverage and negative results

The ordinary kernel was inspected around coefficient normalization, exact-weight grouping, sparse arithmetic, forward expansion, finite model parsing, inverse dispatch, Lagrange coefficients, frontier construction, public object assembly, residual checks, and numerical dispatch. The series-operation layer was inspected across arithmetic, composition, and differentiation. Dedicated source reads covered the numerical evidence and special-function adapter modules, the rational certificate module, Fourier inversion, reciprocal-log calculus, source-coordinate reconstruction, exact-core perturbation, flat operations, Gamma/Barnes inverse construction, Gamma forward construction, and Dirichlet special functions. Relevant Mathics adapters were read for assumptions, simplification, Taylor expansion, numerical roots, time budgets, held calls, core functions, and list operations.[9, 13, 11, 10, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22]

This breadth matters because a suspected local defect may be repaired later in the load sequence. For example, the general Mathics time-budget adapter scales internal proof allowances, but the late core-function adapter restores the native time constraint at the user-selected core-check and callable-application sites. That candidate was discarded, not reported as a fresh defect. Similarly, the inspected certificate source checks the source predicate over the entire closed interval; N1 does not claim that check is absent.[24, 25, 11]

No additional performance defect was isolated with enough new evidence to avoid repeating the prior reports. Nor did this review obtain measurements that would support a new package-wide performance claim. Section 6 instead analyzes the concrete costs and limitations of the proposed branch-reference implementation. Unexamined modules and unrun paths have no implied clean bill of health.

3 The contracts that meet at these two boundaries

3.1 A formal inverse, a numerical root, and an error estimate are different objects

For the ordinary finite model, the repository works in a positive local coordinate u and normalizes the forward expression into a leading monomial with higher-weight corrections. In the simple case relevant here,

$$f(x) = x + a_2x^2 + a_3x^3 + \cdots, \qquad x \to 0^+,$$

the inverse germ is

$$g(y) = y - a_2y^2 + (2a_2^2 - a_3)y^3 + \cdots.$$

An exclusive cutoff of 2 retains the finite expression y. That is a correct statement about a germ at the endpoint, not a claim that y approximates every positive solution of $f(x) = y$. The kernel's normalized source-weight cutoff and retained-block assembly implement that convention.[9]

A numerical root finder has another task: obtain some solution of the equation from its starting data. Official `FindRoot` documentation states that a single starting value uses Newton methods; it does not endow the numerical solution with a symbolic inverse-germ identity. In the current checker the starting point comes from the approximation itself. This is a useful candidate generator, but it makes an independent branch check particularly important.[26, 10]

Finally, a number labeled as an approximation error needs a reference to the same mathematical object. Let $\widetilde{g}(y)$ be the finite approximation and let $r(y)$ be a candidate root. The quantity

$$|r(y) - \widetilde{g}(y)|$$

is an equation-root discrepancy. It equals the selected inverse error only after $r(y) = g(y)$ has been established on the intended branch. A small residual, high precision, and a positive source coordinate establish none of that identity by themselves.

3.2 An equation transformation also transforms its derivative

The special-inverse adapter retains both an original equation and a transformed equation. For the two relevant families these are

$$F(x) = o + s\log\Gamma(x) \quad \text{or} \quad F(x) = o + s\Gamma(x), \qquad s \neq 0,$$

and the transformed equation uses $\log\Gamma(x)$. The adapter correctly records the scope of its finite forward model and several remainder bounds in terms of that transformed equation. It then exposes a derivative-bound key whose name suggests the original equation, without applying the corresponding scaling/Jacobian factor.[12]

A derivative bound is not merely an expression. Its minimum useful meaning contains the function differentiated, the differentiation variable, the domain, whether the bound is signed or an absolute magnitude, and whether it is pointwise or uniform on an interval. N2 is a concrete instance where omitting those distinctions creates incompatible interpretations of otherwise useful mathematics.

4 N1: equation-root agreement can select the wrong inverse

4.1 Source trace

The relevant public entry is `InverseNumericalCheck`. Ordinary inverse objects reach `numericalInverse Evidence` in `src/Kernel/NumericalInverseChecks.wl`. The source path has the following logic: validate the requested/input precision; evaluate the target-domain predicate; numerically evaluate the finite approximation; reconstruct a source seed for a nonunit observable power; call an unbracketed `FindRoot`; verify a real source point on the selected source side; verify the retained source-domain predicate; and compare that root or its observable with the approximation.[10, 9]

The domain helper `inverseEvidenceSourceDomain` normalizes a stored source predicate from either the source variable or a legacy local coordinate. It does not manufacture a connected monotonic interval joining the result to the endpoint. If the stored predicate is merely true or $x > 0$, multiple roots on that side remain admissible. That is a source fact, not a conjecture about numerical conditioning.[11]

The returned association contains a reference root, an approximation, an error, residuals, and a source-domain verification flag. Its text distinguishes numerical evidence from interval certification, which is appropriate. The remaining gap is that "numerical, not certified" is not the same distinction as "equation root, but unverified inverse branch." N1 asks the API to make the latter explicit.

4.2 An exact cubic witness

Consider

$$f(x) = x + 8x^2 - 16x^3. \tag{1}$$

The source-predicted construction is

```
s = AsymptoticInverse[
  x + 8 x^2 - 16 x^3, {x, 0}, {y, 2}];
Normal[s]   (* predicted: y; native call unexecuted here *)
InverseNumericalCheck[s, 1/2, WorkingPrecision -> 50]
```

The inverse coefficients begin $1, -8, 144$, so the finite cutoff-2 expression is indeed y. At $y = 1/2$, the root equation factors exactly:

$$f(x) - \frac{1}{2} = \left(x - \frac{1}{2}\right)(1 - 16x^2). \tag{2}$$

Its real roots are $-1/4$, $1/4$, and $1/2$.

Proposition 4.1. *The positive inverse germ of* (1)*, continued on the increasing source component incident to* 0*, takes the value* $g(1/2) = 1/4$*. The finite approximation takes the value* $1/2$*, which is another exact positive root. Their discrepancy is* $1/4$*.*

Proof. The derivative is $1 + 16x - 48x^2$. It is concave and has positive endpoint values 1 and 2 on $[0, 1/4]$, hence is positive throughout that interval. Thus f increases from 0 to $1/2$ there. Its inverse on that interval is the continuation of the germ selected at 0^+. Equation (2) gives the value $1/4$. The other statements follow by exact substitution. □

The approximation is therefore an especially bad source of an *independent* reference in this example: the seed $1/2$ already has residual zero. If the root solver returns that unchanged exact-root seed, the current postconditions admit it. It is real, positive, solves the stored original equation, and satisfies an unrestricted positive-side source predicate. The resulting discrepancy is zero, whereas the selected-branch error is $1/4$.

This review did not execute that `FindRoot` call. The mathematical witness and the insufficiency of the acceptance predicates are exact; the statement about the displayed public association is a source prediction conditional on the ordinary zero-residual-seed behavior. The supplied characterization program records the actual result in a suitable native kernel rather than hard-coding it as an observation.

4.3 Why checking the derivative sign is not enough

A tempting repair is to require the derivative at the numerical reference to have the same sign as the leading branch. The following polynomial defeats that repair:

$$q(x) = 384x^4 - 304x^3 + 56x^2 + x. \tag{3}$$

Again the cutoff-2 inverse expression is y, and

$$q(x) - \frac{1}{2} = \frac{(2x-1)(4x-1)(8x-1)(12x+1)}{2}. \tag{4}$$

The roots are $-1/12$, $1/8$, $1/4$, and $1/2$.

Proposition 4.2. *For* (3)*, the selected positive branch gives* $g(1/2) = 1/8$*. The seed* $1/2$ *is an exact positive root with* $q'(1/2) = 21 > 0$*, agreeing in derivative sign with* $q'(0) = 1$*. Nevertheless the selected-branch error of the finite approximation is* $3/8$*.*

Proof. Write $x = z/8$ on $[0, 1/8]$. The cubic $q'(z/8)$ has Bernstein coefficients

$$1, \qquad \frac{17}{3}, \qquad \frac{67}{12}, \qquad \frac{15}{4}$$

in the basis $\binom{3}{k} z^k (1-z)^{3-k}$, $0 \le k \le 3$. Every basis function is nonnegative and the basis sums to one, so the derivative is strictly positive throughout this interval. The factorization identifies its endpoint image as $1/2$. Direct differentiation gives the derivative at the other root. Subtraction gives $1/2 - 1/8 = 3/8$. □

The repair must identify a connected component or a proved continuation path. Pointwise derivative orientation at a remote root is not a substitute. This quartic control is useful precisely because it prevents a superficially plausible but inadequate fix from passing the regression suite.

4.4 A scalable exact family

The examples are not tied to the decimal size $1/2$. For rational $t > 0$ and rational $\lambda > 1$, define

$$f_{t,\lambda}(x) = x + \frac{\lambda^2}{t} x^2 - \frac{\lambda^2}{t^2} x^3. \tag{5}$$

Then

$$f_{t,\lambda}(x) - t = (x - t)\left(1 - \frac{\lambda^2 x^2}{t^2}\right).$$

The positive roots at target t are t/λ and t. The derivative is concave on the real line and is positive at the endpoints of $[0, t/\lambda]$, taking values 1 and $2(\lambda - 1)$. The selected germ gives t/λ, while the leading approximation gives the other exact root t. The true error is

$$t\left(1 - \frac{1}{\lambda}\right).$$

The independent tests use four rational targets and three rational values of λ, producing twelve exact subcases. In particular, a fixed heuristic such as "accept any sufficiently small numerical target" cannot solve branch identification across arbitrary inputs. This family changes the forward function with t; it does *not* prove failure arbitrarily close to the endpoint for one fixed analytic function. Nor does it assert a uniform asymptotic theorem in an unbounded parameter family.

4.5 What the witness does and does not invalidate

There is no contradiction in the formal asymptotic expansion. Its finite leading approximation is intentionally low order, and an asymptotic remainder does not promise accuracy at every admitted point. The defect is the strength a user may attach to the numerical comparison after the checker has accepted its reference.

The following are separate assertions:

A: $f(r) = y$,
B: r is real and satisfies the stored source predicate,
C: r lies on the branch continuing the selected inverse germ,
D: $|r - \widetilde{g}(y)|$ measures the error of that inverse.

The current source checks numerical versions of A and B. D requires C as well. The exact examples show that A and B, even supplemented by a derivative sign, do not imply C.

A certificate proving that a bracket contains a unique root proves uniqueness *in that bracket*. It need not prove connection to the asymptotic endpoint unless the certificate's hypotheses include that connection. This report does not relabel such a local root certificate as a false theorem. It recommends keeping the two proof scopes distinct.

4.6 A conservative repair with a precise scope

A low-risk API repair is to expose a branch-reference status. If only the equation and source predicate have been checked, return an explicitly unverified branch status and call the difference a candidate-root discrepancy. Keep the numerical root and residuals useful; do not throw them away solely because they are not yet an inverse reference.

For a verified path, accept a retained branch interval or identify an endpoint-incident monotonic component. A caller-supplied interval must not automatically mean "the selected germ"; its endpoint connection needs evidence. The proposed bounded polynomial oracle makes that requirement executable.

Theorem 4.3 (Endpoint-component reference)**.** *Let $f \in \mathbb{R}[x]$ be nonconstant, let c be a finite source endpoint, and choose a side $\sigma \in \{-1, 1\}$. Let J be the open interval from c toward that side ending at the nearest real zero of f' strictly on that side, or at the corresponding infinity if none exists. For a target $y \neq f(c)$, any root $r \in J$ of $f(r) = y$ is unique in J and belongs to the inverse branch incident to c on that side.*

Proof. The derivative has no zero in J and therefore has constant nonzero sign there. The restriction $f|_J$ is strictly monotone and continuous, so it has a unique continuous inverse on its image. Its endpoint limit is c as the target tends to $f(c)$ from the image side. Thus a root in J is precisely the endpoint-incident inverse value. □

The supplied `endpoint_branch_reference` implements this theorem for $\mathbb{Q}$-polynomials with rational c and rational targets. It isolates real algebraic critical points and equation roots exactly, then admits the unique root in J. The comparison path has no floating-point fallback. An undecidable comparison or an exhausted supported domain produces a refusal.

The stopping rule is conservative. A zero of f' that does not reverse monotonicity still ends this implementation's component. For example, a branch through a stationary inflection may continue, but the prototype refuses to claim that continuation. It also excludes symbolic parameter families, transcendental forward functions, infinite source endpoints, and unspecified input remainders. These are explicit limits of this reference tool, not claims that the repository lacks all such capabilities.

4.7 An immediate user-level control

For the cubic witness, the interval $0 < x < 1/3$ contains the desired branch point $1/4$ and excludes the remote root $1/2$. The following is an unexecuted native control:

```
s = AsymptoticInverse[
  ConditionalExpression[x + 8 x^2 - 16 x^3, 0 < x < 1/3],
  {x, 0}, {y, 2}];
InverseNumericalCheck[s, 1/2, WorkingPrecision -> 50]
```

The existing source-domain check should refuse the remote root rather than claim a selected-branch comparison. This is not expected to make the current root finder automatically find 1/4.

An independent exact bracket is $[7/32, 9/32]$. Elementary interval arithmetic bounds the derivative below by

$$1 + 16\frac{7}{32} - 48\left(\frac{9}{32}\right)^2 = \frac{45}{64} > 0,$$

and exact endpoint signs bracket the root. Together with the separately proved increasing connection from 0, this identifies the intended point. The archive checks those rational inequalities; it does not claim to have run the repository's interval certificate on them.

5 N2: the derivative-bound field names the wrong scope

5.1 The precise source mismatch

The function `specialGamma` constructs adapters for `"Gamma"` and `"LogGamma"`. The wrapper permits an exact real target offset and any provably nonzero real target scale. The stored `Function` includes that affine target transformation. The stored `ExactTransformedFunction` is `LogGamma[x]`, and the forward-model scope correctly says it is the transformed target equation.[12]

In the same object, the property `OriginalDerivativeLowerBound` is set to

$$B(x) = \log x - \frac{1}{2x} - \frac{1}{12x^2}, \tag{6}$$

independently of the family and target scale. This is a useful lower bound for $\psi(x)$, the derivative of $\log\Gamma(x)$ on the positive axis. It is not generally a lower bound for the derivative of the object's stored original `Function`.

A charitable possible meaning of "original" is "the exact LogGamma function, as opposed to its truncated Stirling model." Under that interpretation the expression is mathematically appropriate, but the property name and neighboring `Function` field invite a different interpretation. A scoped replacement resolves both the naming ambiguity and the otherwise false original-equation bound. No inference about the author's intended meaning is needed to demonstrate the ambiguity.

5.2 A counterexample wholly inside the recorded source domain

Take the admitted option `"TargetScale"->1/2` in the LogGamma family. The original equation is

$$F(x) = \tfrac{1}{2}\log\Gamma(x), \qquad F'(x) = \tfrac{1}{2}\psi(x).$$

The adapter's source restriction is $x > 2$, so $x = 3$ is an interior test point. The expressions there are

$$B(3) = \log 3 - \frac{19}{108}, \qquad F'(3) = \frac{1}{2}\left(\frac{3}{2} - \gamma\right).$$

The digamma value follows, for example, from its standard integral/recurrence identity.[27]

Proposition 5.1. *At $x = 3$, the property value $B(3)$ is strictly greater than the actual positive derivative $F'(3)$. Thus it is neither a signed lower bound nor a magnitude lower bound for the half-scaled original equation.*

Proof. The positive atanh series for $\log 3$ gives $\log 3 > 1$, and Euler's constant is positive. Hence

$$B(3) > 1 - \frac{19}{108} = \frac{89}{108} > \frac{3}{4} > \frac{1}{2}\left(\frac{3}{2} - \gamma\right) = F'(3).$$

Positivity of $F'(3)$ also follows from the lower-bound proof below. □

The independent 80-decimal mpmath sample is

$$\begin{aligned} B(3) &\simeq 0.92268636274218376546931 9311, \\ F'(3) &\simeq 0.46139216754923356969674 3955, \\ \tfrac{1}{2}B(3) &\simeq 0.46134318137109188273465 9655. \end{aligned}$$

These decimals corroborate the exact argument; they are not interval certificates or observed Wolfram outputs.

Nor can a larger unspecified asymptotic threshold repair the unscaled interpretation. Since $B(x)/\psi(x) \to 1$, the ratio $B(x)/(\psi(x)/2)$ tends to 2, not to a value below one. For a negative target scale, the signed derivative also reverses direction. For the Gamma family, the original derivative contains the additional factor $\Gamma(x)$.[28]

5.3 Why the transformed bound is valid

The source formula itself is useful and should be retained. For real $x > 0$, the digamma integral representation is[27]

$$\psi(x) = \log x - \frac{1}{2x} - 2\int_0^\infty \frac{t\,dt}{(t^2 + x^2)(e^{2\pi t} - 1)}.$$

The integral is positive. Replacing $1/(t^2 + x^2)$ by the strict upper bound $1/x^2$, and using

$$\int_0^\infty \frac{t\,dt}{e^{2\pi t} - 1} = \sum_{n \geq 1} \frac{1}{(2\pi n)^2} = \frac{1}{24},$$

gives $B(x) < \psi(x)$. Moreover

$$B'(x) = \frac{1}{x} + \frac{1}{2x^2} + \frac{1}{6x^3} > 0,$$

and $B(2) = \log 2 - 13/48 > 1/2 - 13/48 > 0$. Therefore

$$0 < B(x) < \psi(x), \qquad x > 2. \tag{7}$$

The admitted source domain is exactly strong enough for a positive derivative-magnitude lower bound after the appropriate transformation.

5.4 Correct transport and explicit quantifiers

For the adapter's original function, the chain rule gives

$$|F'(x)| \geq \begin{cases} |s|B(x), & F(x) = o + s\log\Gamma(x), \\ |s|\Gamma(x)B(x), & F(x) = o + s\Gamma(x), \end{cases} \qquad x > 2. \tag{8}$$

The derivative sign is $\operatorname{sign}(s)$ in both cases. The target offset contributes no derivative factor. The lower bounds in (8) are pointwise in the original source variable; they do not by themselves supply a constant valid on a requested interval.

For a closed interval $I \subset (2, \infty)$, a usable constant is any proved μ satisfying

$$0 < \mu \leq \inf_{x \in I} \big(|s| B(x)\big)$$

in the LogGamma case, with the corresponding Gamma factor in the other case. Substituting the center into a pointwise bound is not a general way to establish this infimum. The opt-in helper explicitly records that limitation.

5.5 Narrow metadata repair

The supplied `ScopeGammaAdapterDerivativeBounds.wl` does not patch repository definitions or change an expansion. It accepts an existing package-created Gamma/LogGamma special-inverse object, returns a new object, removes the ambiguous old key, and adds a scoped contract. Its principal fields are

```
"TransformedDerivativeLowerBound" -> B[x]
"DerivativeBoundContract" -> <|
  "Type" -> "PointwiseMagnitudeLowerBound",
  "Function" -> originalFunction,
  "Variable" -> x,
  "Domain" -> assumptions && sourceDomain && x > 2,
  "LowerBound" -> Abs[targetScale] familyJacobian B[x],
  "DerivativeSign" -> Sign[targetScale],
  "TransformedFunction" -> LogGamma[x]
|>
```

Here the family Jacobian is 1 for LogGamma and $\Gamma(x)$ for Gamma. The helper has not been executed in a Wolfram or Mathics kernel. Its file passed only a lexical delimiter check; that is not a language-parser or integration result. The independent tests prove the chain-rule identities symbolically and check numerical samples of the transported bound.

For upstream integration, the smallest clean change is to emit these scoped fields directly in `specialGamma`. A compatibility migration must make a deliberate choice about the old property: remove it with a release note, or retain it temporarily with an unmistakable transformed-equation scope. Merely multiplying the expression while preserving an undocumented signed-versus-magnitude interpretation would leave part of the ambiguity unresolved.

5.6 Consequences that are not claimed

This review has not shown that `SpecialInverseNumericalCheck` uses the misleading property to bound its error. The inspected numerical path solves the stored exact transformed equation. It has also not shown that an existing special-function interval certificate consumes this property. The ordinary rational certificate machinery is not being accused of differentiating the wrong equation on the strength of this field.[12, 11]

The confirmed issue is narrower and still worth fixing: a public object's natural derivative-bound interpretation is false for admitted options. That defect can mislead downstream user code even before the repository itself has a consumer. The fix preserves the valid transformed bound and makes the intended downstream contract explicit.

6 Implementation, cost, and UX tradeoffs

6.1 What the branch reference implementation actually does

The Python oracle first admits a narrow exact input class. It accepts a symbolic expression, a source symbol, a finite rational endpoint, a rational target, and a side. It rejects floating-point coefficients, unbound parameters, nonpolynomial operations, invalid options, endpoint targets, and inputs exceeding declared structural budgets. It is a library function, not a text-evaluation service for untrusted strings.

Before polynomial expansion, a recursive syntactic degree bound rejects excessive powers and products. Cancellation is not used to rescue an over-budget expression. The default limits are degree 32, rational numerator/denominator bit lengths 4096, and 2000 input operations. Exact degree and coefficient sizes are checked again after polynomial construction. These are prototype admission policies, not measured optimal settings and not wall-clock or memory guarantees.

The admitted polynomial's derivative roots determine the open endpoint component. The target equation's exact real roots are then filtered against that interval. SymPy's algebraic root objects are retained instead of forcing large radical formulas. All comparisons must resolve exactly; no numerical epsilon determines component membership. The result includes the component endpoints, the selected reference root, the derivative sign, every real equation root, and all same-side roots.

The oracle deliberately returns more information than a single root. In the quartic example this makes the ambiguity inspectable: three roots lie on the positive source side, but only the first is in the endpoint-incident critical-point-free component. That explanation is a more useful UI outcome than a generic statement that the numerical approximation was poor.

6.2 Costs of exact branch identification

For degree d, the reference implementation isolates roots of two polynomials, of degrees at most $d-1$ and d. It then performs a finite set of algebraic order tests. The subsequent selection cost is small compared with isolation; it would be misleading to describe the whole algorithm as a linear-time root check. Coefficient height, multiplicity, and root separation can substantially affect exact isolation work.

For repeated targets of the same forward polynomial and endpoint, the derivative roots and component can be retained once. This is a specific reuse opportunity for the new branch-reference data, not another general repository-wide caching recommendation. Target roots still need to be located or enclosed. A production Wolfram path could use its existing exact polynomial root machinery or an already proved monotonic interval rather than depending on Python.

The prototype enumerates all target roots because it is an independent audit oracle and because negative controls benefit from seeing rejected alternatives. A production path may instead isolate only within the retained component. An approximate candidate can still accelerate that search, but correctness must not depend on its basin of attraction. These are algorithmic tradeoffs, not freshly measured regressions in the current repository.

6.3 Refusal is informative when its scope is precise

A target at the first critical value, beyond the conservative component, or outside the image of that component produces a refusal. Such a result must not be reported as "the equation has no real root." The quartic has several roots precisely when the selected reference needs care. Appropriate distinctions are:

No equation root in the admitted component; branch connection unverified; or reference outside the supported input class.

The attached oracle expresses these restrictions in its exception messages and result scope. A repository integration should retain the same distinction in structured failure/status data.

There is also a useful nonbreaking transition path. Existing numerical consumers can continue to receive candidate roots and residuals. Consumers that require inverse errors can request or inspect a selected-branch status. The strengthened branch-negative tests allow either the independently identified reference or an explicit unverified/refused result. They do not force a general-purpose branch solver into every numerical call.

6.4 The bound repair should not pretend to be an interval prover

The metadata helper changes only the interpretation and transport of an existing bound. It does not compute an interval infimum, verify a branch continuation, prove uniformity in symbolic parameters, or certify the numerical inverse. Those tasks require additional input and remain outside the helper's scope. Keeping it narrow makes the proposed repair easy to review: the finite expression, remainder, and stored original equation must remain unchanged.

The desired-contract tests include both half scaling and negative scaling, the Gamma Jacobian, and object-preservation controls. Testing only the default positive unit scale would miss the concrete defect. These option-sensitive tests are attached to N2 rather than presented as a generic new testing architecture.

7 A focused development plan

This plan is limited to the new findings. It does not repeat the repository's existing transseries, refinement, native-adapter, resource-budget, or Mathics expansion roadmap.

Stage	Change	Acceptance condition
A: interpretation	Distinguish equation-root discrepancy from selected-inverse error; scope the derivative field.	Existing valid candidate-root use remains available. An unverified branch is not silently described as an identified inverse reference. The old derivative interpretation is no longer ambiguous.
B: bounded reference	Add endpoint-component admission for exact polynomial inputs or retained proved monotonic intervals.	Cubic and quartic controls identify 1/4 and 1/8, respectively, or explicitly refuse/unverify; neither accepts 1/2 as the selected-germ reference.
C: option transport	Emit transformed and original-magnitude derivative contracts in the special adapter.	Half-scale LogGamma and negative-scale Gamma bounds include the required factors, while the finite expansion and remainder remain identical.

Stage	Change	Acceptance condition
D: retained evidence	Reuse the selected polynomial component across target checks and invalidate it when its defining source data change.	The returned component refers to the actual forward equation, source endpoint, and side; old component data are not applied to a changed polynomial.
E: broader paths	Only then extend branch-reference evidence to other inverse kinds, using their existing branch representations.	Every new path states how it identifies the selected branch; a shared result head alone is not accepted as that proof.

The most important regression is the quartic one. It protects against a repair that checks only the derivative sign, and it should remain even after the simpler cubic case passes. The scaled cubic family protects against magnitude heuristics. Source reflection, source translation, and target-sign reversal check that a component-based implementation uses the actual coordinate geometry rather than assuming positive coefficients.

For the derivative field, release acceptance should include inspection of the serialized property meanings, not only equality of displayed formulas. The same expression $B(x)$ is valid for one equation and invalid for another; a test that ignores the equation label can pass while the contract remains misleading.

The Mathics port needs a particularly careful interpretation of N1 tests. Its current numerical adapter can refuse unavailable root precision before a noninteger-seed witness reaches a returned numerical root. Such a refusal is not evidence that branch identification is implemented. Conversely, failure of that runtime-specific numerical capability is not a new issue in this report. The exact polynomial oracles and source-level branch obligation are runtime-independent.[23, 22]

8 Artifacts and reproducibility

8.1 Included files

Artifact	Meaning
`article/article.tex`, `article/article.pdf`	This source-linked article and compiled document.
`code/branch_reference.py`	Executed, bounded-input exact polynomial reference oracle and a small independent reversion oracle. Not an upstream package replacement.
`code/test_review.py`, `code/run_review.py`	The 28 independent test methods and evidence generator.
`code/ScopeGammaAdapterDerivativeBounds.wl`	Unexecuted opt-in metadata repair for existing special-inverse objects.
`code/native_characterization.wl`	Unexecuted native observation recorder, including the cubic/quartic and source-domain controls.
`code/native_regressions.wlt`	Seven unexecuted desired-contract/positive-control tests.

Artifact	Meaning
`evidence/independent-results.json`, `evidence/python-tests.txt`	Actual Python-run results and transcript, with method/subcase counts separated.
`evidence/novelty-ledger.json`, `evidence/source-map.json`, `evidence/scope.json`	Exclusion reasoning, pinned source loci, and execution/coverage limitations.
`README.md`, `requirements.txt`, `Makefile`, `LICENSE`	Reproduction instructions, tested Python dependencies, article build, and license for original audit material.

No checksum files are included. The commit identifier is source provenance, not a checksum manifest. The archive does not contain a full upstream checkout or any redistributed font files.

8.2 Independent execution

From the archive root:

```
python -m pip install -r requirements.txt
python code/run_review.py
```

The second command regenerates the evidence JSON and transcript. Preserve the supplied evidence separately when comparing later runs. The final run recorded 28 methods, zero failures, zero errors, and zero skips. Its exact algebraic controls include the factorizations, correct branch points, wrong-root residuals, Bernstein positivity, the twelve-member scaled family, coordinate transformations, conservative refusals, and source-bracket inequalities. The derivative tests include exact chain rules and separately identified 80-decimal samples.

During development, an initial test attempted structural simplification of the square of a `CRootOf`. That assertion did not normalize to the expected integer. It was replaced with an exact minimal-polynomial and positivity check; the branch-reference algorithm did not change for that correction. The evidence records this distinction rather than misclassifying a test-oracle normalization issue as an upstream defect.

8.3 Native characterization and desired contracts

In a fresh Wolfram kernel, load the selected repository source before parsing the audit scripts. A typical interactive sequence is:

```
Get["/absolute/path/to/Asymptotic/src/Kernel/AsymptoticAnalysis.wl"];
$AuditOutputFile = "/absolute/path/to/native-observations.json";
Get["/absolute/path/to/audit/code/native_characterization.wl"];

Get["/absolute/path/to/audit/code/ScopeGammaAdapterDerivativeBounds.wl"];
TestReport["/absolute/path/to/audit/code/native_regressions.wlt"]
```

These commands are supplied instructions, not a run log. The observation recorder names the expected commit but does not independently verify the loaded source identity. The user must select the pinned checkout. It records actual values rather than declaring the source prediction to be a pass.

The desired-contract branch tests accept a refused/unverified branch or the independently correct root. Separate positive controls require the expected leading finite expression; broad refusal alone therefore cannot substitute for all functionality. A failure of a desired-contract test on the unchanged baseline is an expected characterization possibility, not automatically a test-harness defect.

8.4 Build and visual checks

The article uses ordinary LaTeX packages and can be built with `makepdf`. The delivered PDF was compiled, rendered, and visually inspected. Those document checks concern layout and completeness, not mathematical or native-code certification. The Python and Wolfram artifacts retain their separate execution labels throughout the bundle.

9 Conclusion

A careful incremental review should not turn an extensive historical backlog into a fresh defect count. The new contribution here is narrower: *identify the inverse before interpreting a root discrepancy, and identify the equation before interpreting a derivative bound.*

The first obligation survives exact arithmetic and a zero residual. The cubic proves the basic failure of same-side admission; the quartic proves that derivative orientation at a single point does not repair it. A conservative critical-point component gives an executable, bounded reference strategy with explicit refusal semantics.

The second obligation survives arbitrarily large source arguments. The unscaled digamma bound is valid and useful, but its natural original-equation interpretation fails for admitted target scaling. A scoped chain-rule contract preserves the useful mathematics without claiming an interval certificate or modifying the expansion.

Both improvements can be integrated independently of the symbolic coefficient engines. Their native acceptance remains to be run; the source reasoning, exact witnesses, independently executed reference code, and concrete regression specifications are supplied now.

A Pinned source loci and coverage limits

The following source links are fixed to the reviewed revision. Function names are the primary locators, so the report remains interpretable even when a reader compares a later file with different line numbers.

Source	Inspected responsibility and limitation
`src/Kernel/NumericalInverseChecks.wl`	Entire short module; `numericalInverseEvidence` and `numericalSourceDomainCheck`. Primary N1 source.
`src/Kernel/SpecialFunctionAdapters.wl`	Constructor, option admission, `specialGamma`, and numerical route. Primary N2 source.
`src/Kernel/InverseCertificates.wl`	Rational enclosure operations, source-domain normalization and closed-interval admission, residual/derivative proof path, and accuracy machinery. No claimed fresh execution.

Source	Inspected responsibility and limitation
`src/Kernel/AsymptoticAnalysis.wl`	Major core sections around normalization, sparse algebra, forward/model/inverse construction, assembly, residual/numerical dispatch, and loading. Not every line claimed read.
`src/Kernel/SeriesOperations.wl`	Broad arithmetic/composition/differentiation inspection. No new unduplicated defect asserted.
`src/Kernel/FourierCoefficients.wl`, `ReciprocalLogOperations.wl`	Scale-specific algebra, remainder metadata, and residual/calculus paths. No fresh runtime or performance assertion.
`src/Kernel/CorePerturbation.wl`, `SourceCoordinates.wl`, `FlatSectorOperations.wl`	Selected construction and operation sections; explicit carriers, domains, and error transport. Partial module coverage.
`src/Kernel/GammaForward.wl`, `GammaInverse.wl`, `BarnesInverse.wl`, `DirichletSpecialFunctions.wl`	Dedicated asymptotic construction and bound formulas; not all companion/check/operation files inspected.
Mathics assumption, simplification, Taylor, numerical, time-budget, core-function, call, and list adapters	Relevant definitions and late rewriting interactions. No Mathics package execution or claim of complete compatibility.
Maintained review registers and wave indexes	De-duplication and current-status context. Not 36 full article rereads or re-executions.
Repository tests, build, and CI	Assessed only where source/documentation surfaced relevant context. No complete checkout-wide test inspection, full suite run, benchmark, or fresh CI result.

References

[1] Vladimir Reshetnikov, *Asymptotic*, commit `8e859961d7d37f008b826f3a8cad406460271614`. Immutable commit metadata.

[2] Existing code-review collection; see also the pinned wave-1, wave-2, and wave-4 indexes.

[3] Maintained implementation and review-status crosswalk.

[4] Wave-3 review intake.

[5] Wave-4 review intake; wave-4 catalog and evidence distinctions.

[6] Review 13, original article, inspected opening and novelty discussion concerning numerical error resolution.

[7] Review 17, original article, inspected opening and summary concerning resolved numerical discrepancies and certificate scope.

[8] Review 28, original article, inspected findings and novelty discussion concerning Mathics numerical precision.

[9] Main modular kernel, especially `construct`, `inverseDispatch`, `InverseNumericalCheck`, and inverse-object assembly.

[10] General numerical inverse evidence, especially `numericalInverseEvidence`.

[11] Exact interval evidence, especially `inverseEvidenceSourceDomain`, `certPositiveCondition`, and `certAttempt`.

[12] Special-function inverse adapters, especially `specialGamma`, `specialConstruct`, and `specialNumerical`.

[13] General series operations.

[14] Fourier-polynomial coefficient inversion.

[15] Reciprocal-logarithmic calculus.

[16] Source-coordinate construction and reconstruction.

[17] Exact-core perturbation construction.

[18] Flat-sector operations.

[19] Gamma inverse construction; Gamma forward construction.

[20] Barnes inverse construction.

[21] Dirichlet special-function adapters.

[22] Maintained Mathics compatibility guide. Historical receipts are not fresh execution in this review.

[23] Mathics numerical precision adapter.

[24] Mathics internal time-budget adapter.

[25] Late core-function and user-deadline preservation adapter.

[26] Wolfram Research, *FindRoot*, official Wolfram Language documentation, consulted 10 September 2026. `https://reference.wolfram.com/language/ref/FindRoot.html`.

[27] NIST Digital Library of Mathematical Functions, §5.9, particularly equations 5.9.15–16, integral representations of the digamma function. `https://dlmf.nist.gov/5.9`.

[28] NIST Digital Library of Mathematical Functions, §5.11, equation 5.11.2, digamma asymptotics. `https://dlmf.nist.gov/5.11`.